



ANKARA YILDIRIM BEYAZIT UNIVERSITY
FACULTY OF ENGINEERING & NATURAL SCIENCES
SUMMER PRACTICE REPORT

STUDENT'S

NAME SURNAME: Ahmet Yılmaz
DEPARTMENT: Software Engineering
STUDENT ID: 220501234567
E-MAIL ADRESS: ahmet.yilmaz@aybu.edu.tr
PHONE NUMBER: 0532 000 00 00

Contents

1	DAILY REPORT	1
1.1	Week 1	1
1.2	Week 2	2
1.3	Week 3	3
1.4	Week 4	4
1.5	Week 5	5
2	GENERAL REPORT	6
2.1	Introduction	6
2.1.1	Information about the Institution	6
2.1.2	Overview of the Internship	6
2.2	Internship Description	6
2.2.1	Onboarding and Environment Setup	6
2.2.2	Project 1: <i>Logistics Dashboard UI</i>	7
2.2.3	Project 2: <i>REST API rate limiter</i>	7
2.2.4	Project 3: <i>Unit test coverage push</i>	7
2.2.5	Additional Tasks and Contributions	8
2.3	Conclusion	8
2.4	Appendices	9

1 DAILY REPORT

1.1 Week 1

<i>DAYS</i>	<i>WORK ACCOMPLISHED</i>	<i>WORKING HOURS</i>
<i>Monday</i> <i>26/05/2025</i>	First day on site. HR onboarding, signed NDA, received laptop and credentials for the internal portal.	09:00 – 18:00
<i>Tuesday</i> <i>27/05/2025</i>	Set up local development environment: Node.js, pnpm, Docker, PostgreSQL. Cloned the main monorepo.	09:00 – 18:00
<i>Wednesday</i> <i>28/05/2025</i>	Walk-through of the codebase with the supervisor. Read the architecture document and coding-style guide.	09:00 – 18:00
<i>Thursday</i> <i>29/05/2025</i>	Attended sprint planning. Picked a starter ticket: fix a tooltip alignment bug in the Vue.js dashboard.	09:00 – 18:00
<i>Friday</i> <i>30/05/2025</i>	Opened my first pull request. Two rounds of code review, then merged. Wrote a short retro note.	09:00 – 18:00

Signature of trainee :

Work place :

Name and title of the controlling superior :

Signature and Stamp :

1.2 Week 2

<i>DAYS</i>	<i>WORK ACCOMPLISHED</i>	<i>WORKING HOURS</i>
<i>Monday</i> <i>02/06/2025</i>	Kicked off Project 1 (Logistics Dashboard UI). Read existing component code and sketched the new layout.	09:00 – 18:00
<i>Tuesday</i> <i>03/06/2025</i>	Built the new <code>ShipmentTable</code> Vue component using Composition API and TypeScript. Added basic props.	09:00 – 18:00
<i>Wednesday</i> <i>04/06/2025</i>	Wired the component to the existing REST endpoint. Handled loading and empty states.	09:00 – 18:00
<i>Thursday</i> <i>05/06/2025</i>	Pair-programmed accessibility fixes (key-board navigation, ARIA labels) with the senior frontend dev.	09:00 – 18:00
<i>Friday</i> <i>06/06/2025</i>	Public holiday (Eid al-Adha / Kurban Bayramı) – office closed.	—

Signature of trainee :
Work place :
Name and title of the controlling superior :
Signature and Stamp :

1.3 Week 3

<i>DAYS</i>	<i>WORK ACCOMPLISHED</i>	<i>WORKING HOURS</i>
<i>Monday</i> <i>09/06/2025</i>	Bayram tatili devam (half-day return). Caught up on Slack and code-review backlog.	13:00 – 18:00
<i>Tuesday</i> <i>10/06/2025</i>	Finished Project 1. Opened final PR, addressed review comments, merged into main.	09:00 – 18:00
<i>Wednesday</i> <i>11/06/2025</i>	Started Project 2 (REST API rate limiter). Read the Node.js / Express service code and existing middleware.	09:00 – 18:00
<i>Thursday</i> <i>12/06/2025</i>	Researched token-bucket vs sliding-window algorithms. Chose sliding-window log with Redis as the backing store.	09:00 – 18:00
<i>Friday</i> <i>13/06/2025</i>	Implemented a first draft of the limiter middleware. Wrote unit tests with Jest and ioredis-mock.	09:00 – 18:00

Signature of trainee :
Work place :
Name and title of the controlling superior :
Signature and Stamp :

1.4 Week 4

<i>DAYS</i>	<i>WORK ACCOMPLISHED</i>	<i>WORKING HOURS</i>
<i>Monday</i> <i>16/06/2025</i>	Code review feedback on Project 2: refactored to extract config (limits per route) into env variables.	09:00 – 18:00
<i>Tuesday</i> <i>17/06/2025</i>	Added logging and Prometheus metrics for blocked requests. Verified with a local Grafana dashboard.	09:00 – 18:00
<i>Wednesday</i> <i>18/06/2025</i>	Wrote integration tests using a real Redis container via testcontainers. Fixed two race conditions.	09:00 – 18:00
<i>Thursday</i> <i>19/06/2025</i>	Merged Project 2. Started Project 3: increase unit-test coverage on the billing module from 62% to >80%.	09:00 – 18:00
<i>Friday</i> <i>20/06/2025</i>	Mapped uncovered branches in the billing module. Wrote tests for the invoice-generation pure functions first.	09:00 – 18:00

Signature of trainee :
Work place :
Name and title of the controlling superior :
Signature and Stamp :

1.5 Week 5

<i>DAYS</i>	<i>WORK ACCOMPLISHED</i>	<i>WORKING HOURS</i>
<i>Monday</i> <i>23/06/2025</i>	Wrote tests around the discount-calculation logic. Learned to mock the date with @sinonjs/fake-timers.	09:00 – 18:00
<i>Tuesday</i> <i>24/06/2025</i>	Added tests for the PDF-invoice service using dependency-injected fakes. Coverage now at 74%.	09:00 – 18:00
<i>Wednesday</i> <i>25/06/2025</i>	Refactored a tangled function to make it testable. Coverage reached 81%. Opened PR with the report.	09:00 – 18:00
<i>Thursday</i> <i>26/06/2025</i>	Code reviews for two teammates. Updated the internal onboarding wiki with the dev-setup steps I had to figure out.	09:00 – 18:00
<i>Friday</i> <i>27/06/2025</i>	Final day. Knowledge-transfer session, signed exit checklist, collected the evaluation form from my supervisor.	09:00 – 17:00

Signature of trainee :
Work place :
Name and title of the controlling superior :
Signature and Stamp :

2 GENERAL REPORT

2.1 Introduction

2.1.1 Information about the Institution

ACME Yazılım A.Ş. is a mid-sized software company headquartered in the Bilkent technopark in Ankara. The company builds business-to-business SaaS products for the logistics sector, including a shipment-tracking dashboard, a route-optimisation engine and a customer-facing portal used by Turkish forwarders and a handful of European partners. Their main customers are medium-sized freight operators that need a modern web interface on top of their existing ERP systems.

The engineering organisation has around forty people split across three product squads, a platform team and a small data team. Each squad owns its services end-to-end and follows a two-week Scrum cadence with daily stand-ups, mid-sprint refinement and a Friday demo. The stack is centred on Vue.js 3 with TypeScript on the frontend, Node.js (NestJS and plain Express) on the backend, PostgreSQL as the primary database, Redis for caching and queues, and AWS (ECS Fargate, S3, CloudWatch) for hosting.

The work culture is informal and English-friendly. Code review is mandatory for every change, the trunk is protected, and deployments to staging happen on every merge through a GitHub Actions pipeline. Engineers are encouraged to spend roughly one afternoon per sprint on internal tooling, documentation or learning.

2.1.2 Overview of the Internship

The internship lasted five weeks, from 26 May to 27 June 2025, on the “Dashboard” squad. The role was primarily frontend with light backend work. Over the period I contributed to three deliverables of increasing complexity: a redesigned shipment table in the operator dashboard, a Redis-backed rate limiter for one of the public REST APIs, and a focused push to lift unit-test coverage on the billing module. In parallel I did code reviews, wrote documentation, and shadowed an on-call rotation. By the end of the internship I had eleven merged pull requests and a clear picture of how a small product team ships software in practice.

2.2 Internship Description

2.2.1 Onboarding and Environment Setup

The first week was deliberately light on coding. After HR onboarding and the security briefing I received access to GitHub, Jira, Slack and the internal wiki, and pulled the main monorepo. The dev environment is fully Dockerised: a single `docker compose up` brings up PostgreSQL, Redis, the auth service and the API gateway, after which the Vue frontend runs against them with `pnpm dev`.

The coding style is enforced by Prettier and ESLint with a custom config, plus a strict TypeScript setup with `noUncheckedIndexedAccess` turned on. Every commit goes through a pre-commit hook that runs the linter and the unit tests for the touched files. My supervisor walked me through the architecture document on day three and assigned a small “starter” ticket so I could practice the full PR workflow before being given real project work. Daily stand-ups are at 10:00 and run for ten minutes; refinement and demos are on Wednesday and Friday afternoons.

2.2.2 Project 1: *Logistics Dashboard UI*

The first project was to rewrite the `ShipmentTable` component on the operator dashboard. The legacy version was a large Vue 2 Options-API component that mixed data fetching, sorting and presentation, and was difficult to extend. The goal was to split it into a thin container plus a presentational table built with the Composition API, add proper TypeScript types for the shipment domain model, and improve keyboard accessibility.

I started by reading the existing code and writing down the props and events it actually used, which turned out to be a small subset of what the file exposed. The new component takes a typed `shipments` prop and emits `select` and `sort` events; data fetching lives in a parent container that uses a Pinia store. The main lessons were practical: how to design a component API before writing the implementation, how to keep visual changes invisible to users while changing the internals, and how much accessibility work (focus order, ARIA roles, keyboard handlers) is involved in a “simple” table.

2.2.3 Project 2: *REST API rate limiter*

The second project was to add a rate limiter to the public tracking API, which had started receiving aggressive automated traffic from a single integration partner. After a short investigation I proposed a sliding-window-log algorithm backed by Redis, configurable per route and per API key, returning standard 429 `Too Many Requests` responses with `Retry-After` headers.

The implementation is a thin Express middleware that uses `ioredis` with a Lua script to make the window update atomic. Tests were the hardest part: unit tests with `ioredis-mock` caught logic bugs, but two subtle race conditions only appeared in integration tests against a real Redis container started with `testcontainers`. I learned that “works on my machine” is not a substitute for testing against the real dependency, and that middleware design benefits enormously from being a pure function of `(request, state)` so it can be exercised in isolation.

2.2.4 Project 3: *Unit test coverage push*

The third project was less glamorous but, in hindsight, the most educational. The billing module had drifted to 62% line coverage and the team wanted it back above 80% before a planned refactor. My job was to write the missing tests and report any code that could not reasonably be tested.

I started with the pure functions (discount calculation, tax rounding, invoice numbering) because they are easy and have a high payoff. From there I moved into the side-effecting code, mocking the database with an in-memory repository and the time with `@sinonjs/fake-timers`. One function – the legacy PDF generator – was so tangled with I/O that I had to refactor it into a generator plus a writer before tests became possible; my supervisor reviewed the refactor carefully. Final coverage landed at 81%, and I came away with a much better intuition for what makes code testable: small functions, explicit dependencies, no hidden global state.

2.2.5 Additional Tasks and Contributions

Outside the three projects I did roughly a dozen code reviews for teammates, mostly on the same dashboard codebase. I also rewrote the “first-day setup” page on the internal wiki, because several of the steps that tripped me up turned out to trip up everyone, and shadowed two on-call shifts with the platform engineer to understand how production incidents are handled (paging via PagerDuty, runbooks in the wiki, post-mortems within 48 hours).

2.3 Conclusion

The internship gave me a concrete answer to “what does industrial software actually look like.” On the hard-skills side I now have working experience with Vue 3 + TypeScript, Node.js middleware, Redis, Jest and the basics of an AWS-based deployment pipeline, and a much better grasp of testing as a design tool rather than a chore. On the soft-skills side I learned how to scope a ticket, how to ask for help efficiently in Slack, and how much value a careful code review adds to both reviewer and author.

The connection to the curriculum was direct. The Databases course made the schema discussions readable; Software Engineering made the Scrum cadence and review workflow feel familiar; Operating Systems and Networks helped me reason about the rate limiter and about debugging an HTTP issue end-to-end. If I were to do the internship again I would push earlier for harder backend tickets, take notes more systematically, and start writing tests on day one instead of after the first round of bug reports. Overall it was a useful, demanding and well-supervised five weeks, and I am grateful to ACME Yazılım and to my supervisor for the opportunity.

2.4 Appendices

Appendix A — Architecture Diagram

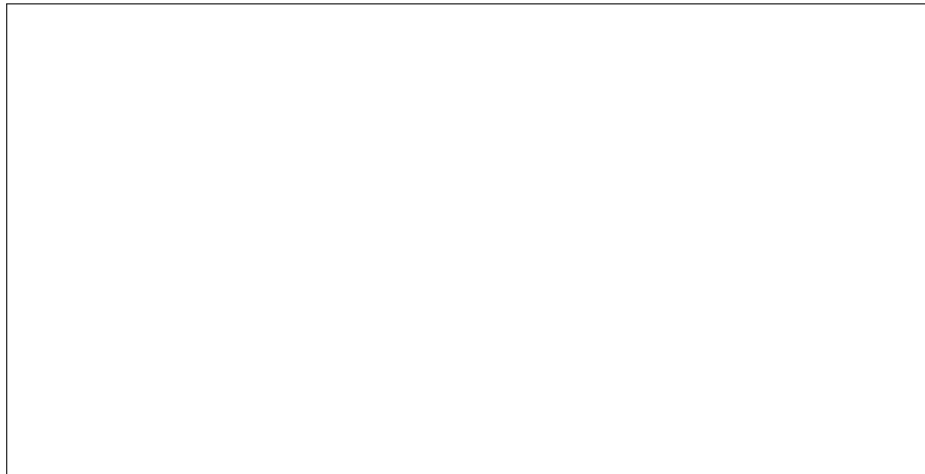


Figure 1: Placeholder — high-level architecture of the ACME logistics platform (frontend, API gateway, services, PostgreSQL, Redis, S3). To be replaced with the real diagram in the printed report.

Appendix B — Screenshots

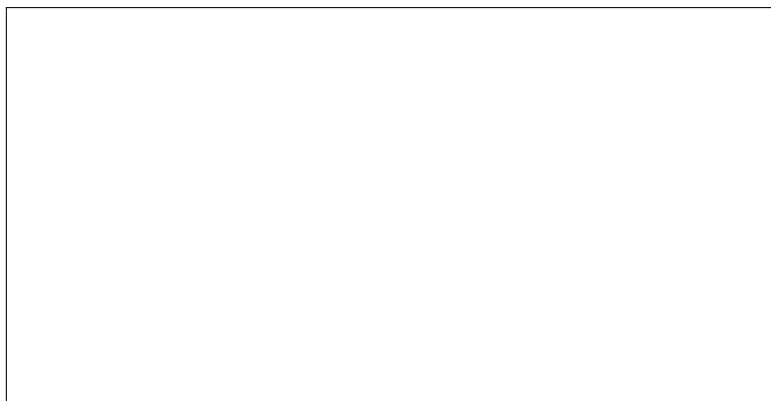


Figure 2: Placeholder — logistics dashboard mockup showing the new `ShipmentTable` component.



Figure 3: Placeholder — Grafana panel showing rate-limited (HTTP 429) responses over a 24-hour window.

Appendix C — Code Listings

```
// Project 2 - sliding-window rate limiter (simplified excerpt).
export function rateLimit(opts: { limit: number; windowMs: number }) {
  return async function middleware(req, res, next) {
    const key = `rl:${req.apiKey}:${req.route.path}`;
    const allowed = await slidingWindow(redis, key, opts);
    if (!allowed) {
      res.set("Retry-After", String(Math.ceil(opts.windowMs / 1000)));
      return res.status(429).json({ error: "rate_limited" });
    }
    return next();
  };
}
```

Appendix D — Links & References

- Internship company: <https://example.com/acme-yazilim> (fictional)
- Internal wiki page authored: <https://wiki.internal/dev-setup>
- Pull requests merged during the internship: 11 (see GitHub profile)